

DAY 4 – DATA VISUALISATION & POWER BI

Beginner Explanation Notes – Based on the Day 4 Notebook Examples and Outputs

Trainer objective: Explain not only how to create a chart, but also what the output means and what business question it answers.

Golden rule: Start with the question → choose the chart → create the visual → explain the insight.

1. Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

We import three libraries. **Pandas** is used for data handling, **Matplotlib** for general-purpose charts, and **Seaborn** for statistical/data-oriented visualisations.

Easy memory: Pandas → data | Matplotlib → graphs | Seaborn → statistical graphs

2. Create the Sales Dataset

```
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 14000, 18000, 21000, 19000],
    "Profit": [2000, 2500, 2200, 3200, 4000, 3500],
    "Category": ["Phone", "Laptop", "Phone", "Laptop", "Tablet", "Laptop"],
    "Region": ["North", "South", "North", "East", "West", "South"]
}
df = pd.DataFrame(data)
df
```

The DataFrame contains 6 rows and 5 columns: Month, Sales, Profit, Category and Region.

Month	Sales	Profit	Category	Region
Jan	12,000	2,000	Phone	North
Feb	15,000	2,500	Laptop	South
Mar	14,000	2,200	Phone	North
Apr	18,000	3,200	Laptop	East
May	21,000	4,000	Tablet	West
Jun	19,000	3,500	Laptop	South

Trainer point: Before creating a visual, students should first understand what each column represents.

3. Understand the Data

df.shape

```
print(df.shape)
# Output: (6, 5)
```

The tuple means **6 rows and 5 columns**.

df.info()

```
print(df.info())
```

This gives the number of rows, column names, non-null information and data types. It helps us check whether the dataset is structured as expected.

df.describe(numeric_only=True)

```
df.describe(numeric_only=True)
```

Important values for this dataset include Sales mean = **16,500**, Profit mean = **2,900**, Sales minimum = **12,000**, and Sales maximum = **21,000**.

Teaching point: describe() gives a quick statistical summary before visualisation.

4. Line Chart – Showing a Trend

```
plt.plot(df["Month"], df["Sales"], marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

The X-axis contains Month and the Y-axis contains Sales. The line connects the monthly values and the marker places a dot at each observation.

Output interpretation: Sales rise from January to May. There is a decrease from February to March and another decrease from May to June.

Remember: Line chart = **Trend**. It is especially useful when values have a meaningful order, such as time.

5. Bar Chart – Comparing Values

```
plt.bar(df["Month"], df["Sales"])
plt.title("Sales by Month")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

The bars allow us to compare the sales value for each month. May has the highest sales (21,000) and January has the lowest (12,000).

Line vs Bar: Line is mainly for trend; bar is mainly for comparison.

6. Histogram – Understanding Distribution

```
plt.hist(df["Sales"])
plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```

A histogram groups numerical values into ranges, called bins. The Y-axis shows frequency: how many observations fall into each range.

Important: A bar chart compares categories or separate values; a histogram shows the distribution of a numerical variable.

7. Scatter Plot – Finding a Relationship

```
plt.scatter(df["Sales"], df["Profit"])
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.show()
```

Each dot represents one row of data. Sales is on the X-axis and Profit is on the Y-axis.

Output interpretation: The points generally move upward as Sales increase, suggesting a strong positive relationship between Sales and Profit.

Remember: Scatter plot = **Relationship**.

8. Seaborn – Why Do We Use It?

```
import seaborn as sns
```

Seaborn is built on top of Matplotlib and works very naturally with Pandas DataFrames. It provides concise syntax for many statistical and categorical visualisations.

Simple comparison: Matplotlib gives general chart control; Seaborn makes many statistical charts convenient and presentation-friendly.

9. Seaborn Style

```
sns.set_style("whitegrid")
```

This changes the visual style of Seaborn charts. It does not create a chart by itself.

10. Seaborn Barplot

```
sns.barplot(data=df, x="Category", y="Sales")
plt.title("Sales by Category")
plt.show()
```

Category is on the X-axis and Sales is on the Y-axis. A Seaborn barplot summarizes the numerical Sales values for each category. With default settings it commonly displays an estimated central value such as the mean.

Remember: Barplot = compare a numerical measure across categories.

11. Countplot

```
sns.countplot(data=df, x="Region")
plt.title("Transactions by Region")
plt.show()
```

Countplot counts how many rows belong to each category. In this dataset: North = 2, South = 2, East = 1, West = 1.

Countplot vs Barplot: Countplot asks “How many records?”; barplot asks “What numerical measure is associated with each category?”

12. Boxplot – Spread and Possible Outliers

```
sns.boxplot(data=df, y="Sales")
plt.title("Sales Boxplot")
plt.show()
```

A boxplot summarises the spread of numerical data. It helps students understand the middle portion of the data, overall spread and possible unusual values/outliers.

Remember: Boxplot = Spread + possible outliers.

13. Heatmap – Correlation

```
corr = df[["Sales", "Profit"]].corr()
sns.heatmap(corr, annot=True)
plt.title("Sales and Profit Correlation")
plt.show()
```

First, corr() calculates the correlation between Sales and Profit. The resulting correlation is approximately **0.993**, which is very close to +1.

This indicates a very strong positive linear relationship in this small dataset: higher Sales values tend to occur with higher Profit values.

Important warning: Correlation does not prove causation. We should say “Sales and Profit are strongly correlated,” not “Sales causes Profit.”

14. Four-Chart Visual Summary

```
fig, ax = plt.subplots(2, 2, figsize=(10, 7))
```

A 2 × 2 subplot creates four plotting areas so that several views of the same dataset can be presented together.

Panel	Visual	Question answered
1	Line	What is the sales trend?
2	Histogram	How are sales distributed?
3	Bar	How do monthly sales compare?
4	Scatter	How are Sales and Profit related?

Why use this? It creates a compact visual summary of trend, distribution, comparison and relationship.

15. Answers to the Notebook Questions

Q1. Which month has the highest sales?

May – 21,000.

Q2. Which month has the lowest sales?

January – 12,000.

Q3. Is profit generally higher when sales are higher?

Yes. The scatter plot shows a strong positive relationship; correlation is approximately 0.993.

Q4. Which category appears most often?

Laptop – 3 records. Phone = 2 and Tablet = 1.

Q5. Which region has the most transactions?

North and South are tied at 2 transactions each. East and West have 1 each.

16. Export Data for Power BI

```
df.to_csv("day4_sales.csv", index=False)
```

This saves the Pandas DataFrame as a CSV file named **day4_sales.csv**. The option `index=False` prevents the DataFrame index (0, 1, 2, ...) from being added as an extra column.

Workflow: Python → Pandas DataFrame → CSV → Power BI → Dashboard

17. Import CSV into Power BI

1. Open Power BI Desktop.
2. Select **Home** → **Get Data** → **Text/CSV**.
3. Choose **day4_sales.csv**.
4. Preview the data and select **Load**.
5. The table and fields appear in the Power BI Data pane.

18. Sales KPI Card

A Power BI Card displays one important business number. Put the Sales field into a Card visual and Power BI can aggregate it as a total.

```
Total Sales
= 12,000 + 15,000 + 14,000 + 18,000 + 21,000 + 19,000
= 99,000
```

For the dataset used in this notebook, the correct total Sales is **99,000**. Depending on formatting, Power BI may display this as approximately **99K**.

19. Sales by Month in Power BI

Use a line chart with **Month** on the X-axis and **Sales** on the Y-axis. It answers: "How are sales changing month by month?"

The highest month is May (21,000), followed by June (19,000).

20. Sales by Category

Use a bar chart with **Category** as the axis and **Sales** as the value. This compares the total sales contribution of Phone, Laptop and Tablet.

21. Sales by Region

Use a bar chart with **Region** as the axis and **Sales** as the value. This helps answer which region contributes more sales.

22. Region Slicer

Add a Slicer and place **Region** into it. Selecting North, South, East or West filters other visuals according to the selected region.

Key Power BI idea: Python charts are mainly analysis outputs; Power BI adds interactive filtering and dashboard exploration.

23. Chart Selection – Quick Revision

Question	Use this chart
How is something changing over time?	Line
Which category/value is bigger?	Bar
How are numerical values distributed?	Histogram
Are two numerical variables related?	Scatter
Are there possible unusual values?	Boxplot
How strongly are variables related?	Heatmap
How many records are in each category?	Countplot
What is one important business number?	KPI Card

24. Final Student Explanation

DATA
↓
Pandas – understand and prepare the data
↓
Matplotlib / Seaborn – visualise the data
↓
INSIGHT – explain what the output means
↓
CSV
↓
Power BI – interactive dashboard
↓
BUSINESS DECISION

One sentence to remember: Python helps us analyse and visualise the data; Power BI helps us communicate the results interactively.

25. Trainer Tips for Beginners

- Do not teach syntax alone. After every chart, ask: “What do you observe?”
- Always connect the chart to a question: trend, comparison, distribution, relationship or count.
- Explain X-axis and Y-axis before discussing the pattern.
- Make students identify the highest, lowest and unusual values from the output.
- Emphasise that correlation is not causation.
- In Power BI, demonstrate the KPI first, then charts, then the slicer.
- For this notebook dataset, remember that total Sales = 99,000.